LESSON 10

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

# Tiny Screen

Write words, draw pictures and make animations on an OLED screen.

Name: _____ Date: _____

### TODAY YOU WILL LEARN

- How to install a **library** (code other people wrote for you)
- How **I²C** lets parts talk over just two wires
- How screen **coordinates** (x, y) work
- How to make an **animation** frame by frame

### SESSION PLAN

| | |
|---|---|
| 0:00 | What's an OLED? Install the library |
| 0:15 | Wire up the screen |
| 0:25 | Sketch 1: **L10_Hello** (text and numbers) |
| 0:50 | Short break |
| 0:55 | Sketch 2: **L10_Shapes** (robot face) |
| 1:20 | Sketch 3: **L10_Bounce** (animation) |
| 1:35 | Challenges |
| 1:55 | Quiz and tidy up |

### PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 0.96" OLED display module (4 pins)
- ☐ 4 jumper wires
- ☐ *For the challenges:* 10 kΩ potentiometer + 3 jumper wires

## 1. A screen for your ESP32

Your kit has a tiny **OLED** screen: 128 dots across and 64 dots down. Each dot is called a **pixel**. "OLED" means every pixel is a tiny light of its own, so it glows brightly on a black background.

Drawing letters dot by dot would be a LOT of work. Luckily, somebody already wrote that code and shared it as a **library**. We just have to install it once.

> **Big idea: libraries**
>
> A **library** is a package of ready-made code. You add it to your sketch with `#include`. Then you can use its commands, like `display.println("Hi")`, without knowing how they work inside.

**Install the library** (once per computer, and your teacher may have done it already):

- ☐ In the Arduino IDE, open **Tools** → **Manage Libraries…**
- ☐ Search for **Adafruit SSD1306**
- ☐ Click **Install**. When it asks about extra libraries it needs, click **Install All**. (This also installs *Adafruit GFX*, which does the drawing.)

## 2. Wire up the screen

The screen talks to the ESP32 using **I²C** (say "I-squared-C"). It needs just two signal wires: **SDA** carries the data, and **SCL** is the clock that keeps time. On the ESP32, I²C always uses pins **21** and **22**.

Read the labels printed on **your** screen's pins. They are not always in the same order! Hold the screen by its pins, not the thin glass.

| ✓ | OLED pin | ESP32 pin |
|---|---|---|
| ☐ | **GND** | **GND** |
| ☐ | **VCC** (or VDD) | **3V3** |
| ☐ | **SCL** (or SCK) | **22** |
| ☐ | **SDA** | **21** |

## 3. Sketch 1: Hello, screen!

Type this in and save it as **L10_Hello**:

Type this in → save it as **L10_Hello** — 26 lines

```
// Lesson 10: Hello, screen!
// OLED screen: SDA on pin 21, SCL on pin 22.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

int count = 0;

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Hello, I am Sparky!");
  display.setTextSize(3);
  display.setCursor(0, 24);
  display.println(count);
  display.display();
  count = count + 1;
  delay(500);
}
```

You should see a message and a number that counts up.

**Screen stays dark?**

Check the four wires, especially that SDA and SCL aren't swapped. Some screens use address **0x3D**: try changing `0x3C` to `0x3D`.

| Code | What it means |
|---|---|
| `#include <Adafruit_SSD1306.h>` | Use the screen library |
| `Adafruit_SSD1306 display(128, 64, &Wire, -1);` | Make a screen called **display**: 128 × 64 pixels, on the I²C wires (`Wire`). The `&` and `-1` are just part of the recipe |
| `display.begin(SSD1306_SWITCHCAPVCC, 0x3C)` | Switch the screen on. **0x3C** is its I²C **address**, like a house number |
| `display.clearDisplay();` | Wipe the picture clean |
| `display.setTextSize(3);` | Make the letters 3 times bigger |
| `display.setCursor(0, 24);` | Start writing at x = 0, y = 24 |
| `display.display();` | **Send** the picture to the screen. Nothing appears until you do this! |

**Big idea: draw, then show**

The library draws your picture in the ESP32's memory first. When the picture is ready, `display.display()` sends all of it to the screen in one go. That's why the screen never shows half-drawn pictures.

**Try it**

1. Change `"Hello, I am Sparky!"` to your own message.
2. Try text size **1**, **2**, **4**. How many big letters fit across?

---

## 4. Sketch 2: Draw with shapes

The screen is like graph paper. **x** counts pixels **across** from the left (0 to 127). **y** counts pixels **down** from the top (0 to 63). Watch out: y goes *down*, not up like in maths class!

```
(0,0) ──────── x ────────▶ (127,0)
  │
  y        (64,32) is the middle
  │
  ▼
(0,63)                    (127,63)
```

Save as **L10_Shapes**:

Type this in → save it as **L10_Shapes** 34 lines

```
// Lesson 10: Draw with shapes
// A robot face that blinks. x goes 0–127 across, y goes 0–63 down.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void drawEyes(int color) {
  display.fillCircle(48, 30, 7, color);
  display.fillCircle(80, 30, 7, color);
}

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.drawRoundRect(24, 10, 80, 53, 8, SSD1306_WHITE);  // head
  display.drawLine(64, 10, 64, 3, SSD1306_WHITE);           // antenna
  display.fillCircle(64, 3, 3, SSD1306_WHITE);
  display.fillTriangle(64, 36, 59, 44, 69, 44, SSD1306_WHITE);
  display.drawRect(46, 50, 36, 7, SSD1306_WHITE);           // mouth
  drawEyes(SSD1306_WHITE);
  display.display();
}

void loop() {
  delay(3000);
  drawEyes(SSD1306_BLACK);  // close the eyes
  display.drawLine(41, 30, 55, 30, SSD1306_WHITE);
  display.drawLine(73, 30, 87, 30, SSD1306_WHITE);
  display.display();
  delay(200);
  drawEyes(SSD1306_WHITE);  // open them again
  display.display();
}
```

| Command | Draws |
|---|---|
| `drawLine(x1, y1, x2, y2, color)` | A line from one point to another |
| `drawRect(x, y, width, height, color)` | A rectangle outline. `fillRect` draws a solid one |
| `drawRoundRect(x, y, w, h, radius, color)` | A rectangle with rounded corners |
| `fillCircle(x, y, radius, color)` | A solid circle centred on (x, y). `drawCircle` draws just the outline |
| `fillTriangle(x1, y1, x2, y2, x3, y3, color)` | A solid triangle with 3 corners |
| `SSD1306_BLACK` | Drawing in black **rubs out**. That's how the eyes close |

> **Plan on paper**
>
> Draw your picture on squared paper first and write down the numbers for each corner.

## 5. Sketch 3: Bouncing ball

An animation is just lots of pictures shown quickly, one after another. Each picture is called a **frame**. Every frame we: move the ball a little → wipe the screen → draw the ball → show it.

Save as **L10_Bounce**:

Type this in → save it as **L10_Bounce** — 32 lines

```
// Lesson 10: Bouncing ball
// The ball moves a little every frame and bounces off the edges.

#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

const int R = 4;  // ball radius
int x = 20;
int y = 10;
int dx = 2;  // how far it moves across each frame
int dy = 1;  // how far it moves down each frame

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  x = x + dx;
  y = y + dy;
  if (x <= R || x >= 127 - R) {
    dx = -dx;
  }
  if (y <= R || y >= 63 - R) {
    dy = -dy;
  }
  display.clearDisplay();
  display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
  display.fillCircle(x, y, R, SSD1306_WHITE);
  display.display();
  delay(10);
}
```

| Code | What it means |
|---|---|
| `x = x + dx;` | Move the ball **dx** pixels across (and `y + dy` moves it down) |
| `x <= R \|\| x >= 127 - R` | `\|\|` means **or**: the ball is touching the left edge *or* the right edge |
| `dx = -dx;` | Turn around! 2 becomes –2 and –2 becomes 2 |

### Try it

1. Make the ball **bigger** by changing `R`.
2. Make it **faster**. Which two numbers do you change?
3. Delete `display.clearDisplay();`. What happens, and why?

---

## 6. Challenges

### Challenge 1: Your own picture

Change L10_Shapes to draw something of your own: a house, a cat, a rocket, or your initials in shapes.

### Challenge 2: Knob meter

Connect a potentiometer (outer legs to 3V3 and GND, middle leg to pin **34**). Show the knob position as a number from 0 to 100% **and** as a bar that grows. *Hint:* `map(analogRead(34), 0, 4095, 0, 100)` *and* `fillRect`.

### Challenge 3: Two balls

Add a second ball with its own `x2`, `y2`, `dx2`, `dy2` that bounces at a different speed.

### Bonus: Paddle game

Draw a paddle along the bottom that you move with the knob. If the ball hits the paddle it bounces up and you score a point. If it misses, the score goes back to 0!

## 7. Quiz

### Check what you learned

1. What does a library do for you?

---

2. Which pixel is at the **bottom-right** corner of the screen?

---

3. Why does nothing show up if you forget `display.display();`?

---

**Tidy up:** unplug the USB cable. Leave the screen wired up, because Lesson 11 uses it to build a weather station!